Lab Sheet 4: SQL Operators

SQL code with sample output (MySQL 8.0 compatible, except INTERSECT/EXCEPT notes)

Sample tables used

The examples use these tables and sample records. Run this setup first in MySQL.

```
CREATE DATABASE IF NOT EXISTS sql_lab4;
USE sql_lab4;

CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  student_name VARCHAR(40),
  course VARCHAR(30),
  city VARCHAR(30),
  marks INT
);

INSERT INTO Students VALUES
(1, 'Aman',  'BTech', 'Jaipur', 82),
(2, 'Neha',  'BCA',   'Delhi',  75),
(3, 'Ravi',  'BTech', 'Jaipur', 68),
(4, 'Priya', 'BTech', 'Agra',   91),
(5, 'Karan', 'BCA',   'Mumbai', 59),
(6, 'Isha',  'MCA',   'Delhi',  NULL);

CREATE TABLE Employees (
  employee_id INT PRIMARY KEY,
  employee_name VARCHAR(40),
  department VARCHAR(30),
  city VARCHAR(30),
  salary DECIMAL(10,2),
  bonus DECIMAL(10,2) NULL
);

INSERT INTO Employees VALUES
(101, 'Amit',  'IT',      'Jaipur', 50000, 5000),
(102, 'Sana',  'HR',      'Delhi',  42000, NULL),
(103, 'Vikas', 'Sales',   'Mumbai', 35000, 3000),
(104, 'Meera', 'IT',      'Agra',   60000, 7000),
(105, 'Rohit', 'Finance', 'Jaipur', 28000, NULL),
(106, 'Anu',   'HR',      'Delhi',  45000, 2500);

CREATE TABLE Applicants (
  person_name VARCHAR(40),
  city VARCHAR(30)
);
INSERT INTO Applicants VALUES ('Aman', 'Jaipur'), ('Neha', 'Delhi'), ('Kabir', 'Pune');
CREATE TABLE Enrolled (
  person_name VARCHAR(40),
  city VARCHAR(30)
);
INSERT INTO Enrolled VALUES ('Neha', 'Delhi'), ('Riya', 'Lucknow'), ('Kabir', 'Pune');
```

A. Arithmetic Operators

1. Addition using +

```
SELECT 12 + 8 AS addition;
```

Sample output:

```
addition
--------
20
```

2. Subtraction using -

```
SELECT 20 - 7 AS subtraction;
```

Sample output:

```
subtraction
-----------
13
```

3. Multiplication using *

```
SELECT 6 * 5 AS multiplication;
```

Sample output:

```
multiplication
--------------
30
```

4. Division using /

```
SELECT 20 / 4 AS division;
```

Sample output:

```
division
--------
5.0000
```

5. Total amount = Price × Quantity

```
SELECT 250.00 AS price, 4 AS quantity, 250.00 * 4 AS total_amount;
```

Sample output:

```
price   quantity   total_amount
250.00  4          1000.00
```

B. Logical Operators

6. AND: both conditions must be true

```
SELECT employee_name, department, salary FROM Employees
WHERE department = 'IT' AND salary > 50000;
```

Sample output:

```
employee_name   department   salary
Meera           IT           60000.00
```

7. OR: either condition may be true

```
SELECT employee_name, department FROM Employees
WHERE department = 'HR' OR department = 'Sales';
```

Sample output:

```
employee_name   department
Sana            HR
Vikas           Sales
Anu             HR
```

8. NOT: condition is reversed

```
SELECT employee_name, department FROM Employees
WHERE NOT department = 'IT';
```

Sample output:

```
employee_name   department
Sana            HR
Vikas           Sales
Rohit           Finance
Anu             HR
```

9. Students in a course AND city

```
SELECT student_name, course, city FROM Students
WHERE course = 'BTech' AND city = 'Jaipur';
```

Sample output:

```
student_name   course   city
Aman           BTech    Jaipur
Ravi           BTech    Jaipur
```

10. Employees in either of two departments

```
SELECT employee_name, department FROM Employees
WHERE department IN ('IT', 'HR');
```

Sample output:

```
employee_name   department
Amit            IT
Sana            HR
Meera           IT
Anu             HR
```

C. Comparison Operators

11. Equal to =

```
SELECT student_name, marks FROM Students WHERE marks = 82;
```

Sample output:

```
student_name   marks
Aman           82
```

12. Greater than >

```
SELECT student_name, marks FROM Students WHERE marks > 80;
```

Sample output:

```
student_name   marks
Aman           82
Priya          91
```

13. Less than <

```
SELECT student_name, marks FROM Students WHERE marks < 70;
```

Sample output:

```
student_name   marks
Ravi           68
Karan          59
```

14. Greater than or equal >= and less than or equal <=

```
SELECT student_name, marks FROM Students
WHERE marks >= 75 AND marks <= 85;
```

Sample output:

```
student_name   marks
Aman           82
Neha           75
```

15. Not equal <> / !=

```
SELECT student_name, city FROM Students WHERE city <> 'Delhi';
-- MySQL also accepts: WHERE city != 'Delhi';
```

Sample output:

```
student_name   city
Aman           Jaipur
Ravi           Jaipur
Priya          Agra
Karan          Mumbai
```

D. Special Operators

16. BETWEEN for salary range

```
SELECT employee_name, salary FROM Employees
WHERE salary BETWEEN 35000 AND 50000;
```

Sample output:

```
employee_name   salary
Amit            50000.00
Sana            42000.00
Vikas           35000.00
Anu             45000.00
```

17. IN for multiple cities

```
SELECT employee_name, city FROM Employees
WHERE city IN ('Jaipur', 'Delhi');
```

Sample output:

```
employee_name   city
Amit            Jaipur
Sana            Delhi
Rohit           Jaipur
Anu             Delhi
```

18. NOT IN

```
SELECT employee_name, city FROM Employees
WHERE city NOT IN ('Jaipur', 'Delhi');
```

Sample output:

```
employee_name   city
Vikas           Mumbai
Meera           Agra
```

19. LIKE: names beginning with a letter

```
SELECT student_name FROM Students WHERE student_name LIKE 'A%';
```

Sample output:

```
student_name
------------
Aman
```

20. IS NULL and IS NOT NULL

```
SELECT employee_name, bonus FROM Employees WHERE bonus IS NULL;

SELECT employee_name, bonus FROM Employees WHERE bonus IS NOT NULL;
```

Sample output:

```
First query (bonus IS NULL):
Sana   NULL
Rohit  NULL

Second query (bonus IS NOT NULL):
Amit   5000.00
Vikas  3000.00
Meera  7000.00
Anu    2500.00
```

E. Set Operations

21. UNION (removes duplicates)

```
SELECT person_name, city FROM Applicants
UNION
SELECT person_name, city FROM Enrolled;
```

Sample output:

```
person_name   city
Aman          Jaipur
Neha          Delhi
Kabir         Pune
Riya          Lucknow
```

22. UNION ALL (keeps duplicates)

```
SELECT person_name, city FROM Applicants
UNION ALL
SELECT person_name, city FROM Enrolled;
```

Sample output:

```
person_name   city
Aman          Jaipur
Neha          Delhi
Kabir         Pune
Neha          Delhi
Riya          Lucknow
Kabir         Pune
```

23. INTERSECT (where supported)

```
-- MySQL 8.0 does not support INTERSECT in all versions.
-- In PostgreSQL / SQL Server, use:
SELECT person_name, city FROM Applicants
INTERSECT
SELECT person_name, city FROM Enrolled;
```

Sample output:

```
person_name   city
Neha          Delhi
Kabir          Pune
```

24. EXCEPT / MINUS (where supported)

```
-- PostgreSQL / MySQL 8.0.31+ supports EXCEPT.
-- Oracle uses MINUS instead of EXCEPT.
SELECT person_name, city FROM Applicants
EXCEPT
```

```
SELECT person_name, city FROM Enrolled;
```

Sample output:

```
person_name   city
Aman          Jaipur
```

F. Combined Problem

25. Arithmetic + Logical + Comparison + Special operators

```
SELECT employee_name, department, salary, bonus,
       salary + COALESCE(bonus, 0) AS total_pay
FROM Employees
WHERE (department = 'IT' OR department = 'HR')
  AND salary BETWEEN 40000 AND 60000
  AND city IN ('Jaipur', 'Delhi', 'Agra')
  AND salary >= 40000
  AND employee_name LIKE 'A%';
```

Sample output:

```
employee_name   department   salary    bonus     total_pay
Amit            IT           50000.00  5000.00   55000.00

Note: COALESCE(bonus, 0) treats a NULL bonus as zero. With the final LIKE 'A%' filter, Amit is the only matching row.
```

Important notes

• SQL output order is not guaranteed unless ORDER BY is used; sample rows are shown for understanding.

• INTERSECT and EXCEPT/MINUS support depends on the database. Oracle uses MINUS; PostgreSQL supports INTERSECT and EXCEPT. MySQL support varies by version.

• NULL must be tested using IS NULL or IS NOT NULL, not = NULL.

• For reproducible order, add ORDER BY to SELECT queries.